Scene Graph Memory Feasibility and Project Guide

A practical plan for robotic imitation learning

Prepared for Vincent Pun | September 2026

A small memory prototype is a reasonable first target. Reproducing the paper's task-success results depends on obtaining a usable training pipeline, sufficient demonstrations, and a compatible evaluation environment. Establish those dependencies before committing to the full experiment.

Project objective

Replicate one task from Expanding Spatial and Temporal Context for Robotic Imitation Learning With Scene Graphs and test whether remembered objects and their 3D positions improve performance. Keep the experiment focused on the two hypotheses in your proposal. [1, 2]

- H1: Persistent memory improves task success compared with using only currently visible objects.
- H2: Removing 3D object positions reduces success when the task depends on remembered locations.

What is feasible at each level

Level	Evidence produced	Current assessment
Memory prototype	Correct object persistence in recorded episodes	Accessible starting point; execution still needs verification
Offline learning study	Predictions on held-out demonstrations	Conditional on data quality and a working training implementation
Task-success replication	Policies complete tasks in a compatible environment	Conditional on evaluation assets, controllers, data, and protocol

The public release contains sample data and preprocessing code. The inspected tree does not contain a policy-training pipeline or simulator evaluation environment. It also differs from the paper in the visual encoder and tracker. Treat these as concrete dependencies to resolve. [2, 3]

Recommendation: First milestone: one verified episode, one trainable batch, one working learning loop, and a confirmed evaluation route.

Reading guide: system concepts on page 2; hardware on page 3; time budget on page 4; task instructions on pages 5-10; decision rules and sources on pages 11-12.

How the system works

The robot sees only part of its surroundings. Memory preserves information from earlier observations so the policy can use it later. The transformer learns which parts of the available representation help predict actions; it does not itself create an authoritative history of every object.

Stage	What it means	What you should inspect
RGB and depth	RGB supplies appearance; depth estimates distance at image pixels.	The image and depth correspond to the same camera and moment.
Detection and tracking	Detection locates relevant objects. Tracking attempts to preserve identity across frames.	The pineapple remains the same object ID as the camera moves.
Persistent object memory	A record preserves appearance and spatial information from past observations.	An unseen object retains past evidence without using future frames.
Transformer encoder	Object and scene tokens interact through attention to produce a scene representation.	Shapes, padding masks, and the information available in each condition.
Diffusion policy	A learned denoiser turns noisy candidate action sequences into predicted actions.	Action units, normalization, output dimensions, and sampling behavior.
Controller and environment	The controller executes commands; the environment generates the next observation.	Actions cause sensible motion and task success has an explicit definition.

Follow one object through the pipeline

Suppose the robot sees a bowl, then turns toward a pineapple. A useful memory keeps the previously observed bowl location. When the robot later carries the pineapple, its policy can condition its actions on that retained location. If a person moves the bowl while it is unseen, the stored position may become stale. Your diagnostic examples should distinguish correct persistence from incorrect assumptions about an unseen change.

Terms you need before coding

- Episode or demonstration: one complete recorded attempt containing a time sequence of observations and expert actions.
- Token: a numerical vector presented to the transformer. An object token is an encoding of object attributes, not necessarily a word.
- Coordinate frame: the reference axes used to describe a position. Camera coordinates change when the camera moves; a common world or robot-base frame supports spatial comparisons.
- Offline evaluation: compare predictions with recorded expert actions. Closed-loop evaluation: execute actions and observe how they change the next state.
- Ablation: remove a specific component and retrain to test its contribution. Training seed: a setting that controls random initialization and other stochastic operations.

Lower offline action error is a useful diagnostic, but it does not show that the robot completes more tasks. H1 and H2 require closed-loop task-success evidence.

Hardware and software requirements

Use your Mac for reading, coding, inspection, and reports. For the first execution attempt, use a Linux NVIDIA machine to minimize changes to the released CUDA code. The paper reports tabletop training on one RTX 3090. A 24 GB GPU is a practical planning target, not a measured minimum or a guarantee that every configuration fits. [2, 4]

Resource	Recommended starting allocation	Why it is needed
GPU	One NVIDIA CUDA GPU; target about 24 GB VRAM	Perception networks and policy training use GPU memory. Measure peak use before scaling.
System memory	32 GB RAM; 64 GB offers more room	Video decoding, episode arrays, dataloaders, and cached features consume host RAM.
CPU	About 8 modern cores as a planning allowance	Parallel data loading and video preprocessing; not a validated minimum.
Storage	Start with 100 GB free SSD space and ability to expand	Model weights, sample episodes, decoded data, caches, checkpoints, and logs. Full dataset size must be audited.
Operating system	Linux on the compute host	Reduces CUDA and third-party model installation risk. Pin the versions that actually work.
Connectivity	Reliable downloads and remote shell access	Dataset and weight transfers, job monitoring, and copying results back to your Mac.
Physical robot	None for recorded-data work or simulation	Required only for real-world rollout evaluation; access includes a controller and calibrated RGB-D camera.

Choose access before purchasing equipment

Start with a university or lab GPU if one is available. Otherwise, use a short cloud session to measure one episode and one training batch. Keep data on persistent storage and stop idle compute. Hardware acquisition is premature until the evaluation route and measured memory demand are known.

A 12-16 GB GPU may work with smaller batches or models loaded sequentially, but compatibility and memory demand must be tested. CPU-only execution and an Apple Silicon port are separate engineering options, with uncertain runtime and dependency support. They are not the default route for this plan.

Additional requirements for task execution

Original simulation needs the correct robot and task assets, camera setup, low-level controller, action interface, and reset and success code. Installing MuJoCo alone does not provide them. A physical setup additionally needs an available arm, gripper, wrist RGB-D camera, calibrated poses, task objects, and supervised lab access. Treat a new hardware integration as separately scoped work.

Hardware acceptance check: Measure peak VRAM, peak RAM, seconds per frame, seconds per training step, and output bytes per episode. Use those measurements to choose the final machine and storage allocation.

Time budget and dependencies

These are planning estimates for one person with working Python and introductory PyTorch knowledge. Hands-on hours cover reading, implementation, inspection, and ordinary debugging. GPU jobs, downloads, author replies, procurement, and major environment reconstruction add elapsed time. They are not promises about completion dates.

Step	Task	Hands-on hours	Compute or waiting
1	Define scope and learn the method	2-3	None
2	Audit code and request materials	3-5	Author or access delays are unknown
3	Select and verify an evaluation route	2-3	Environment access may delay progress
4	Set up the compute environment	4-8	Downloads and installation
5	Process one complete episode	3-5	Measure episode runtime
6	Validate memory and geometry	4-7	Short diagnostic reruns
7	Build the learning dataset	4-6	Feature preprocessing scales with data
8	Build and test the learning loop	8-14	Measure training and inference runtime
9	Decide feasibility and budget	1-2	None
10	Prepare and train three conditions	6-10	Multiple GPU training runs
11	Evaluate and analyze failures	8-14	Rollout and analysis runtime
12	Write and package the project	8-12	Final reproducibility run

Milestone	Base effort	With 30 percent contingency
Feasibility completed through Step 9	31-53 hours	41-69 hours
Full study through Step 12	53-89 hours	69-116 hours

The full-study estimate includes feasibility; do not add the two totals. Reserve another 10-20 hours if tensors, dataloaders, gradient descent, or camera transforms are unfamiliar. This learning allowance is separate from the 30 percent engineering contingency.

At 10 hours per week, budget roughly 5-7 working weeks for feasibility or 7-12 for the full study with contingency. At 20 hours per week, these become roughly 3-4 and 4-6 weeks. Access delays and long jobs can extend either schedule.

Dependency order: Steps 1-3 establish scope and access. Steps 4-8 prove execution and learning. Step 9 authorizes your own commitment to the experiment. Steps 10-12 produce the evidence and report. You can draft the report and pursue access requests while other work proceeds.

Establish scope and access

Step 1 Define the experiment

Time and hardware: 2-3 hours hands-on | Mac or any laptop

Start when: Begin here.

What you are doing: You are converting a broad research interest into a falsifiable comparison. A task must require information that disappears from view for a memory comparison to be informative.

1. Read the main paper method and experiments, then sketch observations, memory, transformer, policy, and controller. Review Diffusion Policy only far enough to explain training noise and action sampling. [2, 6]

2. Write H1 and H2, the three experimental conditions, one candidate task, and the metric you would use. Identify an actual moment when a relevant object is unseen.

3. Use pineapple-to-bowl for the initial data check. Confirm its relevance to the final hypotheses by inspecting episodes; do not infer that relevance from the task name. [5]

Evidence to save: A one-page scope note with a pipeline sketch, hypotheses, evaluation level, and explicit exclusions.

Watch for: A technically correct memory system can show little benefit on a task solvable from the current image alone.

Step 2 Audit the released materials

Time and hardware: 3-5 hours hands-on | Mac or any laptop

Start when: The target paper and repository are identified.

What you are doing: You are checking whether the required inputs exist and whether the code implements the described method. This prevents training work from depending on unavailable infrastructure.

1. Record the paper version and repository commit. Inventory data, weights, policy code, configurations, checkpoints, environment assets, controller, success evaluator, and license or usage conditions.

2. Mark each item available, verified, missing, or unclear. Ask the authors for missing items and the code version used for the paper results. Include action conventions, train/test partitions, seeds, and trial counts in that request.

3. Maintain a discrepancy log. The inspected release uses CUTIE and DINO ViT-B/16; the paper describes XMem and DINOv2. Resolve the intended replication target before silently changing components. [2, 3]

Evidence to save: A dependency inventory and discrepancy log. Keep correspondence with the project records.

Watch for: Sample preprocessing files do not establish that full demonstrations, training code, or task execution are available.

Step 3 Confirm an evaluation route

Time and hardware: 2-3 hours hands-on | Laptop plus access to the candidate environment

Start when: The inventory identifies possible routes.

What you are doing: You are deciding what evidence the project can produce. Real task success needs actions to change an environment and an objective rule to determine success.

1. Choose original simulation, an existing compatible lab setup, an adapted simulation task, or an offline study. Confirm course acceptance if the outcome changes the proposal.

2. For an available environment, run its supplied expert or checkpoint through reset, observation, action, and termination. Inspect a known success and failure against the evaluator.

3. If this cannot be done with available materials, record the blocker and separately scope reconstruction. The 2-3 hours cover route triage and a supplied smoke test, not building a simulator.

Evidence to save: A verified execution route or an explicit narrower offline scope.

Watch for: An adapted environment requires its own compatible demonstrations; recorded actions do not automatically transfer across controllers.

Run the first episode

Step 4 Prepare a reproducible environment

Time and hardware: 4-8 hours hands-on | Linux NVIDIA host and your Mac

Start when: A short compute session is available.

What you are doing: You are isolating a working set of software versions so a future dependency update does not silently change the experiment.

1. Create a project checkout and isolated Python environment. Record OS, GPU, driver, Python, PyTorch, CUDA compatibility, and repository commit. Check GPU visibility before installing the rest.

2. Follow the pinned README for GroundingDINO, SAM, and CUTIE. Install CUTIE from its intended source repository; the release warns that the same PyPI name refers to a different package. Download the required weights. [3]

3. Run import checks, load each model, and process a single image. Save resolved package versions after success. Record errors and fixes rather than repeatedly replacing the whole environment.

4. Keep data, feature caches, runs, figures, and the report in separate directories. Store manifests in version control; keep large datasets and weights outside Git.

Evidence to save: A setup log, pinned environment, working model-load checks, and measured idle and peak GPU memory.

Watch for: All perception models loaded together may exceed available VRAM. Try sequential loading or feature caching before changing the scientific model.

Step 5 Process one recorded episode

Time and hardware: 3-5 hours hands-on | Same NVIDIA host with SSD storage

Start when: The models load and the sample files are available.

What you are doing: You are testing the entire path from recorded observations to numerical object representations. One complete episode exposes problems that a single frame can miss.

1. Download one pineapple episode first. Check file sizes and confirm Git LFS files are real data. Open its trajectory, depth, and video files; compare sequence lengths and available timing information.

2. Set the task data path to a directory containing only this episode. Run the provided preprocessing entry point from the repository root. [3, 5]

3. Inspect output array shapes and values. Render a short diagnostic video with masks, object IDs, boxes, and position estimates. Keep the original observations alongside derived files.

4. Measure wall time, processed frames, peak VRAM, peak RAM, and output bytes. Repeat only when a fix or a new concern justifies another run.

Evidence to save: One fully processed episode, diagnostic frames or video, a schema note, and a runtime record.

Watch for: A file can exist while containing invalid geometry, empty detections, misaligned streams, or silently truncated data.

First command after installation

```
python droid_hodor_test_cutie.py --config configs/pineapple.yaml
```

This is the released preprocessing command, not a policy training command. Run it only after setting the sample data path and obtaining the dependencies and weights. [3]

How to interpret the first measurement

If 600 frames take 20 minutes, the measured average is 2 seconds per frame. Ten similar episodes would take about 200 minutes before overhead. This is an arithmetic example, not a benchmark of this project. Use your own measured values and retain the hardware and configuration alongside them.

Verify memory and prepare training data

Step 6 Check object identity and geometry

Time and hardware: 4-7 hours hands-on | Laptop for review and NVIDIA host for reruns

Start when: At least one complete episode has processed.

What you are doing: You are testing whether memory means what you think it means. Incorrect camera transforms can create plausible-looking numbers while teaching the policy the wrong locations.

1. Select a small diagnostic set, for example 20-30 frames across two or three episodes. Include camera motion, occlusion, reappearance, visible object motion, and episode boundaries.

2. Compare RGB and depth alignment. Review crop, padding, rotation, camera intrinsics, and the direction of the camera-to-reference transform. Check depth units and centroid outliers.

3. Track a stationary object as the camera moves. In the intended common frame its location should remain reasonably stable. Define a tolerance from the sensor data and task scale rather than inventing a universal threshold.

4. Inspect invisible-object behavior, identity recovery, and reset at each episode. Verify that each stored state uses only observations available up to that time. Save examples of stale positions and identity switches.

Evidence to save: An annotated diagnostic set with pass/fail observations and a list of remaining defects.

Watch for: Do not fill earlier memory states using an object appearance discovered later. That leaks future information into training and evaluation.

Step 7 Build a trustworthy learning dataset

Time and hardware: 4-6 hours hands-on | CPU and SSD for loading; GPU for new features

Start when: The representation passes the diagnostic checks.

What you are doing: You are making batches of observations and expert action windows. The model can learn useful control only if those two streams refer to the correct times and units.

1. Inspect the actual trajectory schema. Document action dimensionality, units, frame, control frequency, gripper convention, and absolute versus relative commands. Verify these from data and controller code, not from variable names alone.

2. Build a dataset loader that maps each time step to its causal memory state, optional robot state, and future action window. Handle windows at episode ends with explicit padding and masks.

3. Split complete episodes into training, validation, and test sets before tuning. Group related scenes or recording sessions where appropriate. Fit all normalization parameters on training data only.

4. Create one batch, print its tensor shapes, and visualize its aligned observations and actions. Check missing values, valid masks, finite numbers, and split membership. Treat the ten released episodes as debugging data until adequacy and diversity are established.

Evidence to save: A data schema, episode split manifest, normalization statistics, and a verified training batch.

Watch for: Random frame splits can put almost identical moments from one episode into both training and test sets, making generalization appear much better than it is.

A useful mental check

For every model input, ask: could the robot have known this value at this instant? For every training target, ask: does it describe the action sequence that starts at the intended instant? These two questions catch many failures before expensive training.

Prove learning and decide feasibility

Step 8 Build the smallest learning loop

Time and hardware: 8-14 hours hands-on | NVIDIA GPU with cached features

Start when: A verified batch and action schema exist.

What you are doing: You are checking that the proposed representation can enter a trainable policy and produce valid action sequences. Diffusion training teaches a network to predict noise added to expert actions; inference uses repeated denoising to sample actions.

1. Use the authors' training implementation if available. Otherwise adapt official Diffusion Policy infrastructure and explicitly label the missing pieces you reconstruct. Keep frozen perception features cached. [6]

2. Implement object and scene token construction, geometry encoding, padding masks, transformer aggregation, diffusion conditioning, action normalization, and inverse normalization. Record a shape trace from input batch to output action window.

3. Run forward and backward passes on a tiny subset. Verify finite loss and gradients, then check that training loss falls substantially. Sample actions and inspect their units, range, and temporal alignment.

4. Save and reload a checkpoint. Evaluate an untouched episode and distinguish fitting the tiny subset from generalization. If an environment is available, perform an initial policy-interface smoke test before launching the final experiment.

Evidence to save: A reloadable checkpoint, training curve, sample predictions, and measured seconds per training step and inference call.

Watch for: Decreasing noise-prediction loss alone does not prove correct action decoding or competent behavior. A broken normalization or controller interface can still invalidate rollouts.

Step 9 Make the feasibility decision

Time and hardware: 1-2 hours hands-on | Laptop

Start when: Steps 1-8 have produced their evidence or documented blockers.

What you are doing: You are deciding which complete project is achievable with the measured resources and accessible materials. This protects the remaining time for a defensible experiment.

1. Review the data, perception, memory, learning, and environment evidence. For each item, record verified, needs repair, or unavailable.

2. Estimate remaining preprocessing, training, evaluation, storage, and human effort from your pilot. Select a hard compute budget and a date to stop adding features.

3. Proceed to task-success replication only with a working evaluation route. If only offline data is accessible, revise the deliverable and claims. If adapting simulation, budget its construction separately and validate the expert before policy experiments.

Evidence to save: A one-page feasibility decision, selected scope, measured resource estimate, and remaining dependency list.

Watch for: Waiting for an author response is not a verified path to missing materials. Set a personal decision deadline and pursue independent work in the meantime.

Budget from measurements

Training GPU hours = number of conditions × training seeds × steps per run × measured seconds per step ÷ 3600. Add preprocessing and evaluation separately. For three conditions and three seeds, there are nine final training runs, plus the pilot.

For example, nine four-hour runs would consume 36 GPU hours. This is an illustrative calculation only. Cloud cost is compute hours × the current provider rate, plus storage and transfer charges; no provider price or measured run duration is assumed here.

Prepare the controlled experiment

Step 10 Train comparable policy conditions

Time and hardware: 6-10 hours hands-on | NVIDIA GPU and persistent checkpoint storage

Start when: The Step 9 decision supports the selected study.

What you are doing: You are isolating whether persistent object records and object coordinates explain a performance difference. A comparison is interpretable only when other important training and evaluation choices stay consistent.

1. Implement the three configurations below and inspect an identical example through each one. Confirm that the intended information is absent in the ablations, including cached and auxiliary inputs.

2. Train each condition separately. For visible-only input, mask or omit invisible records so retained embeddings cannot leak into attention or aggregation. For no-3D input, remove object coordinates through every relevant path; preserve the agreed treatment of other inputs.

3. Keep the data split, perception features, optimization budget, checkpoint-selection rule, and evaluation states consistent. Match capacity where practical and document parameter-count differences.

4. Use a pilot to set the final budget. Plan multiple training seeds; three is a practical starting target, not a guarantee of adequate statistical power. Lock the protocol before evaluating the test set.

5. Record configuration, seed, code revision, data manifest, training curve, best validation checkpoint, and resource usage for every run. Keep a failed-run log with reasons.

Evidence to save: Three tested configurations, complete run records, final checkpoints, and a frozen evaluation protocol.

Watch for: Deleting features only at test time measures sensitivity to an unexpected input change. It is not the same experiment as training a policy without those features.

Condition	Unseen object records	Object 3D coordinates	Comparison
Full memory	Retained	Available	Reference condition
Visible objects only	Unavailable to policy	Available for visible objects	Tests H1
Memory without 3D	Retained	Removed	Tests H2

Decisions to record before testing

- Primary metric and exact task-success rule; include timeout and termination behavior.
- Number of training seeds and evaluation trials, chosen from pilot variability and available resources.
- How evaluation initial states are matched across conditions and how failures are logged.
- How a checkpoint is selected using validation data without choosing the best test result.
- How task and subtask completion will be summarized, including uncertainty and per-seed variation.

A small or inconsistent difference may leave a hypothesis unresolved. Do not add trials only because the current result is disappointing. Report the planned protocol and any later exploratory analyses separately.

Evaluate and prepare the final report

Step 11 Measure performance and failures

Time and hardware: 8-14 hours hands-on | GPU or inference host; compatible simulator or lab

Start when: Final configurations and evaluation protocol are fixed.

What you are doing: You are collecting evidence that addresses the hypotheses and diagnosing what limits the system. The evaluation mode determines what conclusions are justified.

1. For closed-loop evaluation, run the planned trials and save task success, subtask completion, initial-state identifiers, seed, timeout, and failure reason. Record representative videos and preserve raw trial records.

2. Report successes divided by attempts with uncertainty intervals and results per training seed. A Wilson interval can describe a single policy's binomial rollout success under suitable trial assumptions; pooled claims must also account for variability between trained policies.

3. Compare conditions on matched initial states when possible. Avoid treating repeated rollouts from one trained policy as independent model replications. Seek statistical guidance if a formal significance claim is central.

4. For offline work, report held-out prediction metrics, units, and aggregation rules; separate visible and occluded periods. Label them as prediction results and leave task-success hypotheses untested.

5. Categorize failures as detection, identity, geometry, stale memory, action prediction, or controller execution. Use attention maps only as supplementary inspection, not proof of a causal mechanism.

Evidence to save: Raw evaluation records, aggregate tables and figures, uncertainty calculations, and a failure gallery.

Watch for: If an expert or supplied checkpoint also fails, investigate the environment or controller before blaming learned memory.

Step 12 Write and package the project

Time and hardware: 8-12 hours hands-on | Mac or any laptop; compute host for final rerun

Start when: Results and limitations are documented.

What you are doing: You are making the work understandable and reproducible. A successful project can report partial replication or inconclusive results if its methods and evidence are sound.

1. Write the report in this order: question and hypotheses; related work; implemented method; differences from the source; data and protocol; results; failure analysis; limitations and conclusion. Draft the stable sections while training runs.

2. Include a pipeline figure, experimental-condition table, task or prediction results with sample counts, and representative failures. Separate measured results from interpretation.

3. Write setup, preprocessing, training, and evaluation instructions. Include configurations, environment versions, data manifests, seeds, checkpoints or access instructions, and figure-generation scripts.

4. Run the documented workflow from a clean environment or fresh checkout as far as practical. Confirm each final figure regenerates from saved data and verify submission requirements with the course rubric.

Evidence to save: A final report, reproducible code package, result tables and figures, and rollout examples when available.

Watch for: Published or example project papers illustrate possible presentation; they do not establish your course's required length or grading criteria.

Decision rules and practical schedule

Decision point	Continue when	If it fails
After Step 3	A compatible route exists or offline scope is accepted	Time-box missing-material requests; separately scope adaptation
After Step 5	One episode completes with readable outputs	Repair data and setup before processing the full dataset
After Step 6	Memory is causal and geometry is credible	Fix alignment and identity errors before training
After Step 8	Learning, action decoding, and checkpoint reload work	Debug on tiny data before increasing model or dataset size
After Step 9	Scope, compute, and evaluation are achievable	Choose a narrower defensible study or revise the task
Before final testing	Conditions, metrics, and budgets are fixed	Complete the pilot and lock the protocol

Suggested work sessions

Use focused sessions of roughly two to three hours. End each session with a saved artifact, a brief observation, and the next unresolved question. Keep long jobs separate from your active work log so you can estimate progress accurately.

- First sessions: finish scope, inventory, and author questions; identify GPU access and the candidate evaluation route.
- Next sessions: establish one-frame and one-episode execution; inspect memory before scaling.
- Following sessions: prepare the causal dataset and a tiny learning run; write the feasibility decision.
- After the decision: run the controlled study, analyze failures, and finish the report. Reserve the final portion of the schedule for reproducibility and writing.

Work that requires a separate estimate

Additional scope	Why the base estimate does not include it
Rebuilding the original simulator or locomotion controller	May require robotics assets, control expertise, training, and extensive integration.
Collecting a full new demonstration dataset	Requires a successful expert, recording tools, sufficient variation, and data-quality review.
Setting up a new physical robot	Requires hardware access, calibration, controller integration, and lab scheduling.
Porting CUDA code to a different backend	Requires dependency support and correctness and performance checks.
Adding tasks or new architectures	Multiplies data preparation, experiments, and interpretation work.

Keep a small project notebook

For each session, record date, step, active hours, code revision, command or configuration, output location, observation, and next action. For each run, record machine type, elapsed compute, peak memory, and success or failure. This becomes the evidence for your feasibility decision and the methods section of your final report.

Sources and final readiness checklist

Project-specific statements refer to the sources below. Effort estimates, hardware allowances, suggested diagnostics, and decision rules are planning recommendations. No project code was executed to establish hardware minima or runtime benchmarks for this report.

[1] Research Proposal: Local one-page proposal supplied in the project folder. Establishes the one-task replication, H1, H2, and distinction between offline prediction and completed tasks.

Research Proposal.pdf

[2] Qian and colleagues: Expanding Spatial and Temporal Context for Robotic Imitation Learning With Scene Graphs. arXiv 2606.01072, version 2. Method, tasks, and supplementary implementation details.

https://arxiv.org/html/2606.01072v2

[3] Official scene graph repository: README and complete public file tree inspected during planning. Tree SHA 69593b0f886ba6e9f77003176f180a5007bfa0f1. Availability and implementation may change.

https://github.com/jianingq/scene-graph-robot

[4] Released preprocessing implementation: CUDA device selection, RGB-D processing, transforms, and saved representations.

https://github.com/jianingq/scene-graph-robot/blob/main/post_process_scene_cutie.py

[5] Pineapple task configuration: Initial sample task, entities, slots, and preprocessing settings.

https://github.com/jianingq/scene-graph-robot/blob/main/configs/pineapple.yaml

[6] Official Diffusion Policy implementation: Starting point for policy training and evaluation infrastructure if the target policy code remains unavailable.

https://github.com/real-stanford/diffusion_policy

Before you commit to the full study

- I can explain the memory representation and the two comparisons.
- I have a documented data schema, valid calibration, and causal episode splits.
- I have watched memory behavior through occlusion and reappearance.
- I have a working learning loop and measured its resource use.
- I can execute and score the task, or have explicitly accepted an offline scope.
- I know what differs from the paper and how those differences limit the claims.
- I have budgeted the full study, compute time, contingency, and report preparation.

The earliest useful deliverable is the feasibility evidence package. Once that package is complete, you can choose the final project scope based on demonstrated capability and available resources.